

Доклад по проекту ZipGo — мобильное приложение каршеринга

1. Введение

1.1 Идея проекта

(Слайд 1)

ZipGo — мобильное Android-приложение каршеринга, которое позволяет пользователям арендовать автомобили напрямую через смартфон: без очередей, звонков и бумажных договоров.

Пользователь открывает приложение, выбирает автомобиль по фильтрам (марка, класс, кузов, цена, даты), просматривает характеристики, добавляет понравившийся в избранное и оформляет договор аренды в несколько касаний.

1.2 Проблемы и актуальность

(Слайд 2)

Рынок каршеринга в России растёт: по данным BusinesStat, к 2025 году число пользователей превысило **15 млн человек**, а ежегодный прирост составляет ~20%. Тем не менее существующие сервисы имеют ряд нерешённых проблем:

Тратит кучу времени — аренда занимает часы: оформление документов, очереди, ожидание менеджера. ZipGo сокращает это до 2 минут прямо в приложении.

Огромный рынок — 15 млн пользователей каршеринга в России, рост 20% в год. Качественный и удобный инструмент востребован как никогда.

Прозрачная статистика — история поездок, расходы и любимые авто — всё в одном экране. Ни один из популярных конкурентов не предоставляет такой аналитики пользователю.

1.3 Задачи проекта

(Слайд 3)

Перед проектом стояло четыре ключевых технических задачи:

- Реализовать **Clean Architecture** (MVVM + Repository) для масштабируемой и тестируемой кодовой базы
- Создать **интуитивный UI** на базе Material Design 3
- Реализовать **multi-select фильтры** каталога с фильтрацией по 6 параметрам одновременно
- Синхронизировать данные с **бэкендом (Spring Boot)** через REST API

1.4 Существующие аналоги и целевая аудитория

Сервис	Особенность	Ограничение
Яндекс.Драйв	Лидер рынка, широкий автопарк	Только крупные города, нет фильтрации по классу
Delimobil	Доступные цены, гибкие тарифы	Слабая аналитика для пользователя
BelkaCar	Москва, удобное приложение	Только один город, нет истории трат
Citydrive	Хорошая база авто	Устаревший UI, нет статистики поездок

Конкурентное преимущество ZipGo: современный UI на Material Design 3, фильтрация по 6 параметрам, полная история контрактов и детальная аналитика — возможности, которых нет ни у одного из конкурентов в полном объёме.

Целевая аудитория: городские жители России 18–45 лет без личного авто или предпочитающие не использовать его в городе.

- Интернет-пользователей с мобильным доступом: ~100 млн человек
- Текущее число пользователей каршеринга: ~15 млн (BusinesStat, 2025)
- Ежегодный прирост рынка: ~20%
- Целевая аудитория приложения: **10–15 млн человек**

2. Демонстрация проекта

2.1 Функциональные модули

(Слайд 4)

Разработаны следующие модули:

Авторизация — регистрация, вход, хранение токена в DataStore, автологин при повторном запуске.

Каталог — лента автомобилей с карточками (фото, марка, модель, класс, цена), multi-select фильтры, пагинация (Infinite Scroll), pull-to-refresh.

Карточка авто — полные характеристики (VIN, госномер, год, кузов, класс), выбор дат аренды с автоматическим расчётом стоимости, быстрое бронирование.

Избранное — сохранение понравившихся авто, синхронизация с сервером, отдельный экран просмотра.

Аренда — создание контракта, управление статусами, просмотр истории, отмена свайпом.

Профиль — статистика по поездкам, редактирование данных пользователя, смена пароля, выход из аккаунта.

2.2 UI/UX особенности

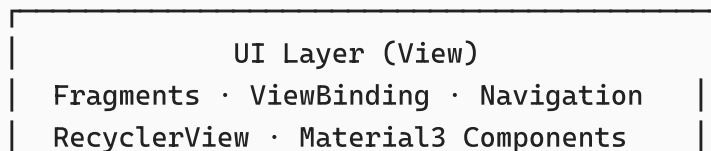
(Слайд 5)

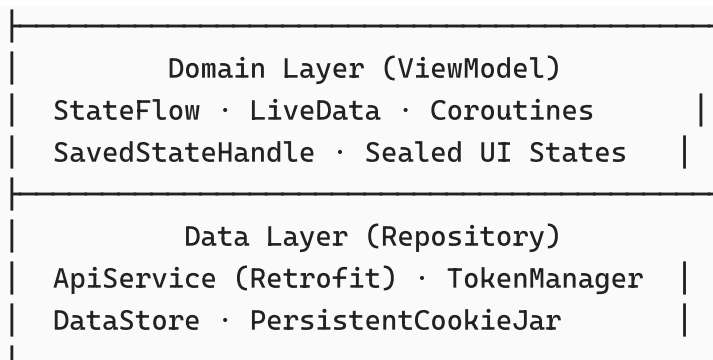
- **Material Design 3** — современные компоненты: MaterialCardView, Chips, BottomSheet, RangeSlider
- **Pull-to-refresh** — обновление каталога жестом
- **Bottom Navigation** — быстрое переключение между четырьмя разделами
- **Fallback-изображения** — 4 встроенных SVG-иллюстрации, выбор по классу авто; один и тот же авто всегда показывает одно изображение (детерминированный выбор по `car.id % 4`)
- **Поддержка светлой и тёмной темы**

3. Описание реализации

3.1 Архитектура проекта

Проект построен по принципу **Clean Architecture** с разделением на три слоя, реализованных паттерном **MVVM + Repository**.





Навигация: единственный MainActivity (Single-Activity Architecture) с Navigation Component и NavGraph. Bottom Navigation управляет переходом между 4 основными разделами. Переходы типобезопасны благодаря Safe Args.

Управление состоянием: MutableStateFlow внутри ViewModel → collect во Fragment через lifecycleScope. Sealed-классы (CatalogueUiState, CarDetailUiState) явно разделяют состояния Loading / Success / Error.

Обработка конфигурационных изменений: SavedStateHandle сохраняет выбранные фильтры, текущую страницу пагинации и состояние UI при повороте экрана.

3.2 Реализация модуля «Каталог»

(Слайд 6)

Фильтры:

- Chips (multi-select) по марке и классу авто
- RangeSlider по диапазону цен и годам выпуска
- Кнопки фильтрации и сортировки в Bottom Sheet
- Добавление в избранное прямо из ленты

Пагинация (Infinite Scroll):

RecyclerView.OnScrollListener отслеживает позицию. Когда до конца списка остаётся ≤ 5 элементов — ViewModel запрашивает следующую страницу (размер страницы: 20 авто). Флаг isLoadingMore предотвращает дублирующие запросы.

Эффективные обновления списка:

ListAdapter + DiffUtil вычисляют минимальный набор изменений в фоновом потоке. Перерисовываются только изменившиеся карточки — UI не блокируется даже при больших списках.

3.3 Реализация модуля «Авторизация»

(Слайд 7)

Вход и регистрация:

- Валидация полей в реальном времени
- Читаемые сообщения об ошибках (неверный пароль, нет сети)
- Сохранение сессии между запусками приложения

Управление сессией:

- Access token хранится в **DataStore** (безопасно, async, нет ANR на main thread)
- **Refresh token** передаётся через HTTP-only cookie, сохраняется в `PersistentCookieJar`
- `AuthInterceptor` перехватывает каждый запрос, добавляет `Authorization: Bearer <token>`. При 401 — автоматически вызывает `/api/refresh` и повторяет оригинальный запрос прозрачно для вызывающего кода
- **Автовход** при перезапуске приложения — токен загружается из DataStore

3.4 Реализация модуля «Профиль»

(Слайд 8)

- **Данные пользователя:** имя, фамилия, e-mail, рейтинг водителя, аватар с инициалами
- **Статистика по поездкам:** общее число поездок, поездки за месяц, сумма трат, любимая марка, средняя стоимость поездки, средняя длительность
- **Редактирование:** имя, фамилия, телефон — через `PATCH /api/profile`
- **Смена пароля** — отдельный экран с валидацией старого и нового пароля
- **Выход** — очистка токена из DataStore и cookie из CookieJar, переход на экран авторизации

3.5 Реализация модуля «Аренда»

(Слайд 9)

Бронирование:

- Выбор дат аренды через `DateRangePicker`
- Автоматический расчёт итоговой стоимости (дни × дневная ставка)
- Создание контракта через `POST /api/contracts`

Статусы контракта:

`PENDING` → `CONFIRMED` → `ACTIVE` → `COMPLETED` — визуальный таймлайн прогресса на

экране контракта. Также поддерживаются `CANCELLED` и `CANCELLATION_REQUESTED`.

Обработка ошибок:

- **409 Conflict** — авто уже занято на выбранные даты: понятное сообщение пользователю
- **Отмена** — свайп по элементу в списке контрактов (swipe-to-dismiss)

3.6 Реализация модуля «Избранное»

(Слайд 10)

- Добавление и удаление одним касанием прямо из ленты каталога
- Отдельный экран для просмотра всех сохранённых автомобилей
- **Синхронизация с сервером:** `POST /api/cars/favorites/{carId}` / `DELETE /api/cars/favorites/{carId}` — избранное не теряется при переустановке или смене устройства
- Быстрый переход к карточке авто из экрана избранного

3.7 Обоснование выбора технологий

Технология	Альтернатива	Причина выбора
Kotlin Coroutines	RxJava	Лаконичный код, нативная поддержка, structured concurrency
Retrofit	Volley, OkHttp напрямую	suspend-функции, автосериализация, гибкая цепочка interceptors
Coil	Glide, Picasso	Kotlin-first, coroutine-aware, меньше boilerplate
DataStore	SharedPreferences	Async API, type-safe, нет ANR на main thread
Navigation Component	ручные FragmentTransaction	Типобезопасная навигация, backstack из коробки
Material Design 3	Custom UI	Соответствие Android HIG, профессиональный вид
ViewBinding	Kotlin Synthetics	Null-safe, compile-time проверки, нет рефлексии

3.8 Интерфейсы между подсистемами

```
Fragment → ViewModel → Repository → ApiService (Retrofit)
                                     ↑
                               AuthInterceptor
                               PersistentCookieJar
                               TokenManager (DataStore)
```

UI ↔ ViewModel: `StateFlow<UiState>`, Fragment подписывается через `lifecycleScope`.

ViewModel ↔ Repository: suspend-функции, Repository возвращает `Result<T>` (Success / Error).

Repository ↔ Backend: интерфейс `ApiService` (Retrofit), все методы — `suspend fun`.

3.9 Use-Case диаграмма

(Слайд 11)

Два актёра: **Клиент** (полностью реализован) и **Администратор** (API готов на бэкенде, UI в разработке).

Клиент: зарегистрироваться / войти, просматривать каталог, применять фильтры, добавлять в избранное, просматривать карточку авто, создавать контракт аренды, просматривать и отменять контракты, просматривать статистику, редактировать профиль.

Администратор (планируется): управлять автомобилями (CRUD), управлять марками и моделями, управлять пользователями, управлять контрактами, просматривать общую статистику парка.

3.10 Диаграммы сценариев

(Слайд 12)

Сценарий авторизации:

```
Пользователь → App: ввод логин/пароль
App → Backend: POST /api/auth
Backend → App: access_token + set-cookie (refresh_token)
App → DataStore: сохранить токен
App → Пользователь: перейти в каталог

При 401 в любом запросе:
App → Backend: POST /api/refresh → новый токен → повтор запроса
```

Сценарий аренды авто:

Пользователь → App: выбрать авто из каталога
App → Backend: GET /api/car/{id}
App → Пользователь: показать детали + ввод дат
App (локально): рассчитать итоговую стоимость
Пользователь → App: нажать «Оформить»
App → Backend: POST /api/contracts
Backend → App: контракт в статусе PENDING
App → Пользователь: экран контракта с таймлайном

3.11 Результаты апробации

- Приложение протестировано на эмуляторах Android API 24–35 (Android 7.0 – Android 15)
- Проверены сценарии: авторизация, пагинация, фильтрация, создание/отмена контракта, редактирование профиля
- Обработаны edge-cases: пустой каталог, нет интернета (код ошибки сети), истёкший токен (автообновление), 409 при конфликте дат
- Минимальный SDK: 24 — охват **~94%** устройств на рынке Android

4. Заключение

4.1 Текущий статус

(Слайд 14)

Реализован полный рабочий прототип приложения с клиент-серверной архитектурой:

- Авторизация с автосохранением сессии и прозрачным обновлением токена
- Каталог с многокритериальной фильтрацией (6 параметров) и бесшовной пагинацией
- Детальный просмотр и бронирование автомобилей с расчётом стоимости
- Управление контрактами: создание, просмотр статусов, таймлайн, отмена
- Избранное с серверной персистентностью
- Профиль с 8 метриками статистики поездок
- Clean Architecture (MVVM + Repository) — код готов к масштабированию
- Поддержка Android 7.0–15 (API 24–35) — охват **~94%** устройств

4.2 Планы по развитию

(Слайд 13)

- **Push-уведомления** об изменении статуса контракта (Firebase FCM)
- **Карта с GPS-геолокацией** ближайших авто (Яндекс.Карты SDK)
- **iOS-версия** (Kotlin Multiplatform Mobile — общий бизнес-слой)
- **Публикация в RuStore**
- Онлайн-оплата (ЮKassa / Stripe)
- Система оценок и отзывов
- Административная панель (UI для управления автопарком)
- Unit/UI тесты (целевое покрытие 80%+), CI/CD на GitHub Actions
- B2B-модуль для корпоративных клиентов